

pypresso: what it computes, and how to ask for it

Plane-wave DFT in Python and JAX, validated against Quantum ESPRESSO

2026-08-26

Contents

pypresso: what it computes, and how to ask for it	1
1. Input and invocation	2
2. Ground state	3
3. Bands, densities of states, projections	4
4. Pseudopotentials and functionals	5
5. Spin, magnetism and spin-orbit	5
6. Forces, stress and relaxation	6
7. Response: dielectric, phonons, Raman	7
8. Topological invariants	8
9. Things with no pw.x counterpart	9
10. Performance: dials, memory and GPUs	10
11. Accuracy summary	11
Where to go next	11

pypresso: what it computes, and how to ask for it

User documentation for the feature set. `README.md` is the short tour and the quickstart this is the reference — every capability, the entry point that reaches it, what it was validated against, and **what it refuses**.

Two conventions run through the whole document and are worth reading first.

Everything is validated against Quantum ESPRESSO on the same input, and the agreement is quoted per feature below rather than claimed in general. Where a number is missing it is because the comparison does not exist, and that is said rather than hidden.

Anything not implemented is refused by name, with an error that says what is missing — never silently approximated or replaced by something close. The “Refuses” lines below are as much a part of the documentation as the features: they are the promise that a run which starts is a run whose physics is there.

One equation is written down and the rest are derived from it. Almost everything below is a derivative of the total energy, taken by the compiler rather than by hand:

$$E[\{\psi\}, \tau, \varepsilon] = T_s + E_H + E_{xc} + E_{\text{loc}} + E_{\text{nl}} + E_{\text{Ewald}}$$

quantity	what it is	how it is obtained
$v_{xc}(\mathbf{r})$	$\delta E_{xc}/\delta n$	jax.grad of the energy density
$\mathbf{F}_I$	$-\partial E/\partial \tau_I$	jax.grad at frozen ψ
σ_{ij}	$-\Omega^{-1}\partial E/\partial \varepsilon_{ij}$	jax.grad along a strain
$C_{I\alpha,J\beta}$	$\partial^2 E/\partial \tau_{I\alpha}\partial \tau_{J\beta}$	jvp of the force
$Z_{I,\alpha\beta}^*$	$\partial F_{I\alpha}/\partial \mathcal{E}_\beta$	jvp of the force along the field response
$\partial \epsilon_{ij}/\partial \tau$	Raman	jvp of the second-order energy

The "frozen ψ " is what makes this exact rather than approximate: at self-consistency the energy is stationary with respect to the wavefunctions, so the terms that would come from $\partial\psi/\partial\lambda$ vanish. That is the Hellmann-Feynman argument, and taking it one order up is the 2n+1 theorem, which is why a *third* derivative needs only first-order wavefunctions.

1. Input and invocation

pypresso reads **pw.x input files**. The same file that runs in Quantum ESPRESSO runs here, which is what makes every comparison in this document possible.

```
from pypresso.io.pwin import read_pw_input
from pypresso.pseudo import read_upf
from pypresso.scf.driver import Calculation, run_scf
from pypresso.system import build_system

system = build_system(read_pw_input("scf.in"))
pseudos = tuple(read_upf(f"pseudo/{s.pseudo_file}") for s in system.structure.species)
result = run_scf(system, pseudos, conv_thr=1.0e-8)

print(result.total_energy, "Ry in", result.iterations, "iterations")
```

conv_thr means what it means in a pw.x input: it is compared against QE's dr2, the Hartree energy of the density residual.

A command line covers the common workflows:

```
python3 -m pypresso.cli inspect <qe-output> # summarise what the parser reads
python3 -m pypresso.cli dos <input> # density of states
python3 -m pypresso.cli pdos <input> # projected DOS
python3 -m pypresso.cli relax <input> # structural relaxation
python3 -m pypresso.cli stress <input> # stress tensor
python3 -m pypresso.cli spiral <input> # spin-spiral scan
```

Units are Rydberg atomic units throughout (Ry, bohr), matching QE. The only layer that speaks anything else is io/ — Hubbard U in the HUBBARD card is in eV and is converted at the boundary, as pw.x does it.

Refuses: K_POINTS gamma selects the half-sphere storage of the gamma-point trick, which is generated but not consumed such a run is substituted by an explicit $k = 0$ on the full sphere — the same physics at twice the storage — and it says so.

2. Ground state

what	entry point
self-consistent total energy, term by term	run_scf, calculation = 'scf'
the converged density, potential, wavefunctions	SCFResult fields
a run continued from another	run_scf(starting_from=result)

SCFResult carries total_energy, energy_terms (QE's own split), eigenvalues, occupations, wavefunctions, density, potential, fermi_energy, homo/lumo, and the magnetization fields when there is one.

Metals are supported with every smearing QE has (gaussian, marzari-vanderbilt, methfessel-paxton, fermi-dirac) and with the tetrahedron family — both the linear and the Blochl-corrected forms — usable as an occupation scheme *inside* the SCF, not only for a density of states.

Continuation across a change of spin regime (starting_from=) promotes a converged state into a target's variables: non-magnetic into collinear, collinear into non-collinear, spin-orbit switched on. It reaches the *same* self-consistent solution as a fresh run ($\leq 4e-8$ Ry on six cases) in as little as one iteration. The magnetization is **seeded** from the target's starting_magnetization when the source has none, because nothing in the SCF breaks spin symmetry on its own and an unseeded promotion would converge back to the unpolarized solution and report success.

Convergence. mixing_mode selects anderson (default), linear or Kerker/Thomas-Fermi preconditioning (TF) scf_solver offers a Newton-Krylov alternative to mixing, which is slower but reaches solutions the mixer cannot hold. max_iterations defaults to 100 — **a hard SCF may need more**, and hitting the cap is reported as not converged rather than as an answer.

The equations

The Kohn-Sham problem, generalised because ultrasoft and PAW make the overlap non-trivial:

$$\hat{H}|\psi_{n\mathbf{k}}\rangle = \epsilon_{n\mathbf{k}}\hat{S}|\psi_{n\mathbf{k}}\rangle, \quad \hat{S} = 1 + \sum_{I,ij} q_{ij}|\beta_i^I\rangle\langle\beta_j^I|$$

$$\hat{H} = -\nabla^2 + v_{\text{loc}}(\mathbf{r}) + v_{\text{H}}[n] + v_{xc}[n] + \sum_{I,ij} D_{ij}^I|\beta_i^I\rangle\langle\beta_j^I|$$

with $n(\mathbf{r}) = \sum_{n\mathbf{k}} w_{\mathbf{k}} f_{n\mathbf{k}} (|\psi_{n\mathbf{k}}|^2 + \sum_{ij} Q_{ij}^I \langle\psi|\beta_i\rangle\langle\beta_j|\psi\rangle)$, the augmentation term being what ultrasoft adds. D_{ij} is rebuilt from the potential every iteration, which is why the SCF is a fixed point in (n, D) and not in n alone.

Convergence is measured as QE measures it — the Hartree energy of the residual:

$$dr^2 = \int \int \frac{\delta n(\mathbf{r})\delta n(\mathbf{r}')}{|\mathbf{r} - \mathbf{r}'|} d\mathbf{r}d\mathbf{r}' < \text{conv_thr}$$

```
result = run_scf(system, pseudos, conv_thr=1.0e-10, mixing_mode="anderson")
print(result.total_energy, result.energy_terms)      # QE's own term-by-term split
print(result.fermi_energy, result.homo, result.lumo)
```

3. Bands, densities of states, projections

what	entry point
band structure on a path	run_bands
non-self-consistent run at fixed density	run_nscf
density of states	run_dos, compute_dos
projected DOS, Löwdin charges, spilling	run_pdos, lowdin_charges

The DOS schemes live behind a name registry (DOS_SCHEMES) — smearing and both tetrahedron variants — and the projected DOS feeds *the same* registry as a per-band weight, so a PDOS and a DOS are the same integration.

Projections are $\langle \phi | S | \psi \rangle$ on Löwdin-orthogonalised pseudo-atomic orbitals, resolved by atom, l and m . Validated against a purpose-built projwfc.x on seven cases: **6.9e-4** on a projection, **4.7e-5** on a Löwdin charge, and the spilling parameter to all four decimals it prints.

The equations

A band structure is one diagonalisation per k on a fixed density a DOS is

$$g(E) = \sum_{n\mathbf{k}} w_{n\mathbf{k}} \delta(E - \epsilon_{n\mathbf{k}})$$

with δ replaced by a smearing function or evaluated by tetrahedron interpolation. The projected DOS weights each term by the overlap with an orthogonalised atomic orbital, $|\langle \phi_\mu | \hat{S} | \psi_{n\mathbf{k}} \rangle|^2$, and the spilling parameter is what is *not* captured: $\sum_{n\mathbf{k}} w_{n\mathbf{k}} f_{n\mathbf{k}} (1 - \sum_\mu |\langle \phi_\mu | S | \psi \rangle|^2)$.

```
from pypresso.workflows.bands import run_bands
from pypresso.workflows.dos import run_dos
from pypresso.workflows.pdos import run_pdos

bands = run_bands(system, pseudos, scf.density, kpoints=path)  # a k-path
dos, _ = run_dos(system, pseudos, scf.density, grid=(12, 12, 12),
                 scheme="tetrahedra_opt")  # returns (DensityOfStates, NSCFResult)
pdos, _ = run_pdos(system, pseudos, scf)  # returns (ProjectedDOS, NSCFResult)
print(pdos.channels[0])  # resolved by atom, l and m
print(pdos.charges.charges, pdos.charges.spilling)  # Lowdin charges, spilling
```

4. Pseudopotentials and functionals

Pseudopotential kinds: norm-conserving, **ultrasoft**, and **PAW** — the two-grid split, the augmentation charge, the overlap operator, self-consistent D_{ij} , and PAW's one-centre terms including its radial Poisson solve and spherical quadrature. UPF v2 is read **UPF v1 is refused**.

Functionals: LDA (PZ), **PBE**, **revPBE**, **PBEsol**, on the plane-wave grid and on the PAW spheres. The functional is taken from the pseudopotential headers unless `input_dft` overrides it, and **an unimplemented one is refused rather than silently replaced by LDA**. QE composes a functional from four independently chosen slots and the UPF headers name all four, so `pypresso` does the same.

Only the **energy** of a functional is written down `v_xc`, and a GGA's `v1` and `v2`, come from `jax.grad` of it.

Meta-GGA, potential-only branch: `input_dft = 'tb09'` (Tran-Blaha) and `'bj06'` (Becke-Johnson). These invert the rule above — they *are* potentials, there is no energy — so the consequences are enforced rather than documented: `run_scf` warns that its total is not the value of any functional it minimised, and **forces, stress, phonons and response all refuse**. Silicon's gap goes from LDA's 0.49 eV to **1.13 eV** against an experimental 1.17. Works with PAW, with noncollinear magnetism and with spin-orbit coupling **plain ultrasoft is refused**, because it has no partial waves to reconstruct τ from inside the sphere.

Refuses: energy-carrying meta-GGAs (TPSS, SCAN, M06L) — their potential has a $dE/d\tau$ piece acting on the wavefunction that is not written EXX.

Van der Waals: Grimme **D2** (`vdw_corr = 'grimme-d2'`). Bilayer graphene matches `pw.x` to **3.1e-9 Ry** and binds at 3.23 Å where PBE alone has no minimum. **D3, Tkatchenko-Scheffler, MBD and XDM are refused by name** — where `pw.x` warns and silently runs with no correction at all.

5. Spin, magnetism and spin-orbit

regime	how to ask	note
unpolarised	default	
collinear	<code>nspin = 2</code> , <code>starting_magnetization</code>	one Fermi level, or two under <code>tot_magnetization</code>
noncollinear	<code>noncolin = .true.</code>	magnetization is a vector magnetic symmetry
spin-orbit	<code>noncolin + lspinorb = .true.</code>	j-resolved projectors, NC/US/PAW

Three spin numbers are kept apart, and the distinction is exposed on `System`: `nspin` says which regime is in force, `npol` is the number of spinor components of a *wavefunction*, and `nspin_mag` the number of components of a *density*. They coincide at 1 and 2 and come apart at 4, where `npol = 2` but `nspin_mag` is **one** for a nonmagnetic spin-orbit run.

Magnetic fields and constraints: a uniform field over the cell, a field inside one atom's sphere (through a `LOCAL_MAGNETIC_FIELDS` card that `pw.x` has no counterpart for), Elk's `reducebf`, and all four of QE's `constrained_magnetization` schemes. **The field's energy is not part of the reported total** — QE prints `etcon` and never adds it, and Elk excludes its external field's energy by the same convention both numbers are carried separately.

DFT+U: QE's `lda_plus_u_kind = 0` (Dudarev) with the J0/beta extension, on atomic, ortho-atomic and norm-atomic projectors, read from the HUBBARD card. Forces through the Hubbard term come from differentiating through projectors that move with the atoms. **Refused:** the full (Liechtenstein) formulation, intersite V, background channels, the orbital-resolved variant, the wf/pseudo projector sets, and noncollinear `ns_nc`.

Refuses: PAW + GGA + a noncollinear magnetization together (PAW_gcxc_potential with `nspin_mag = 4`).

6. Forces, stress and relaxation

what	entry point
forces	<code>compute_forces</code> , <code>tprnfor = .true.</code>
stress and pressure	<code>compute_stress</code> , <code>tstress = .true.</code>
structural relaxation	<code>run_relax</code> , <code>calculation = 'relax'</code>
variable-cell relaxation	<code>run_vc_relax</code> , <code>calculation = 'vc-relax'</code>

The force is `jax.grad` of the total energy at frozen wavefunctions, with the orthonormality constraint carried explicitly so ultrasoft's Pulay term is part of the same gradient. QE's `force_lc`, `force_cc`, `force_ew`, `force_us`, `addusforce` and `force_corr` are transcribed too, behind the same registry (`force_methods`) — **as a cross-check, not as the implementation**. The two share no machinery, which is what makes disagreement informative it is how the augmentation force's sign and a missing gradient correction were found.

Stress is the same construction in the other coordinate: `sigma = -(1/0mega) dE/d(epsilon)`. `stress_methods` offers the analytic terms beside it note `stres_us/addusstress` are **not** transcribed, so the analytic route offers terms and no total.

Variable-cell relaxes the cell and atoms together over `3 nat + 9` coordinates at an applied pressure, minimising the **enthalpy** — so a relaxed crystal carries the applied pressure rather than having no stress. The gap between the frozen-basis energy and a fresh SCF at the relaxed geometry is the Pulay error of the basis and is **reported** (`VCRelaxResult.pulay_error`) rather than left to be noticed `treinit_gvecs` rebuilds the basis per step and makes it zero.

The equations

$$\mathbf{F}_I = -\left. \frac{\partial E}{\partial \tau_I} \right|_{\psi \text{ fixed}}, \quad \sigma_{ij} = -\left. \frac{1}{\Omega} \frac{\partial E}{\partial \varepsilon_{ij}} \right|_{\psi \text{ fixed}}$$

Both are one gradient. With ultrasoft the orthonormality constraint $\langle \psi_n | \hat{S} | \psi_m \rangle = \delta_{nm}$ is carried explicitly, so its multiplier term — QE's Pulay contribution — falls out of the same derivative rather than being added afterwards.

A variable-cell relaxation minimises the **enthalpy**, which is why a relaxed crystal carries the applied pressure rather than having zero stress:

$$H = E + P\Omega, \quad \frac{\partial H}{\partial h} = \Omega(P\mathbf{1} - \sigma)h^{-T}$$

```

from pypresso.forces import compute_forces
from pypresso.stress import compute_stress
from pypresso.workflows.relax import run_relax
from pypresso.workflows.vc_relax import run_vc_relax

forces = compute_forces(calculation, result)           # method=... for QE's terms
print(forces.forces)                                  # (nat, 3) in Ry/bohr
stress = compute_stress(calculation, result, terms=True) # kbar and Ry/bohr^3
print(stress.pressure)

relaxed = run_relax(system, pseudos, forc_conv_thr=1.0e-4)
cell     = run_vc_relax(system, pseudos, press=500.0)   # kbar
print(cell.pulay_error)                                # the frozen-basis error, reported not hidden

```

7. Response: dielectric, phonons, Raman

All of this is built on the **Sternheimer equation** — a projected conjugate- gradient solve per occupied band — rather than a sum over states. There are no empty states to converge and no division by $\epsilon_{\text{ps}_n} - \epsilon_{\text{ps}_m}$.

what	entry point
velocity / position operator	band_velocities, VelocityOperator
dielectric tensor	dielectric_tensor
Born effective charges	born_effective_charges
dynamical matrix, phonons at Γ	dynamical_matrix, Phonons
elastic constants	elastic_constants
electrostriction, elasto-optic	electrostriction
Raman tensors	raman_tensors
Raman/IR spectra, mode activities	vibrational_spectrum, mode_activities

The **perturbations are gradients of code that already exists**: `dv_of_drho` is one jvp of `v_of_rho`, the field's commutator is the velocity operator, `dvqpsi_us` is one jvp through `at_positions`, and `PAW_dpotential` is one jvp of the one-centre terms.

Ultrasoft and PAW work for the dielectric constant ($\leq 1.2e-4$ against `ph.x` on four cases). **Metals work** for the response and the dynamical matrix. Phonons on a symmetry-reduced wedge work as of the rank-n symmetriser.

Refuses, by name: Born charges for PAW (`int3_paw` against `becsumort` is missing, and the expression is wrong in sign as well as size without it) χ^2 and the electro-optic tensor (the second-order source term `dvpsi_e2/solve_e2` has nothing to build the $\langle u_i | r_k | u_j \rangle$ piece from — and it is **42%** of the answer, measured) the non-analytic LO-TO term phonons at $q \neq 0$ the dynamical matrix of an ultrasoft dataset and any response under a potential-only meta-GGA.

A trap worth knowing: a response on a reduced k-set is a *polar vector field* and must be symmetrised as one. Running the whole k-grid instead is only sound if the

grid is closed under the point group — a **shifted** Monkhorst-Pack grid is not, and that combination is refused by name.

The equations

Every first-order quantity comes from the Sternheimer equation — the linear response of an occupied orbital to a perturbation ΔV , projected outside the occupied manifold so no empty states and no energy denominators appear:

$$(\hat{H} - \epsilon_n \hat{S} + \alpha \hat{Q})|\Delta\psi_n\rangle = -\hat{P}_c^+ \Delta V |\psi_n\rangle$$

It is solved self-consistently, because ΔV contains the response of the potential to the response of the density — see below — and the kernel K in it is one jvp of `v_of_rho` rather than a second implementation:

$$\Delta V_{\text{scf}} = \Delta V_{\text{ext}} + \int K(\mathbf{r}, \mathbf{r}') \Delta n(\mathbf{r}') d\mathbf{r}'$$

From its solutions:

$$\epsilon_{\alpha\beta}^{\infty} = \delta_{\alpha\beta} + \frac{8\pi}{\Omega} \sum_{n\mathbf{k}} w_{\mathbf{k}} \langle \Delta_{\alpha} \psi_n | \partial_{\beta} \psi_n \rangle, \quad Z_{I,\alpha\beta}^* = \frac{\partial F_{I\alpha}}{\partial \mathcal{E}_{\beta}}$$

$$C_{I\alpha, J\beta} = \frac{\partial^2 E}{\partial \tau_{I\alpha} \partial \tau_{J\beta}}, \quad \omega^2 \mathbf{e} = \frac{1}{\sqrt{M_I M_J}} C \mathbf{e}, \quad \frac{\partial \epsilon_{ij}}{\partial \tau_{I\gamma}} \text{ (Raman)}$$

The Raman tensor is the second-order energy differentiated once more along a displacement, at *frozen first-order* wavefunctions — the 2n+1 theorem, which is why a third derivative costs one more jvp and not a new solve.

```
from pypresso.response import (dielectric_tensor, dynamical_matrix,
                               raman_tensors, vibrational_spectrum)

eps = dielectric_tensor(calculation, scf.wavefunctions, scf.eigenvalues, scf.density)
print(eps.epsilon) # 3x3, cartesian

ph = dynamical_matrix(calculation, scf.wavefunctions, scf.eigenvalues,
                      scf.density, scf.becsum)
print(ph.frequencies) # cm^-1

raman = raman_tensors(calculation, scf, born_charges=True, keep_internals=True)
spectrum = vibrational_spectrum(calculation, scf) # per-mode Raman + IR activity
```

8. Topological invariants

what	entry point
Berry curvature	<code>run_berry_curvature</code> , <code>berry_curvature</code>

what	entry point
Chern number	chern_number
Z2, Wilson loop / Wannier charge centres	run_z2, wilson_z2
Z2, Fu-Kane parities	fu_kane_z2, parity_z2
Z2 in 3D	run_z2_3d, z2_invariant_3d

Everything is built from one primitive — the overlap of occupied manifolds at neighbouring k-points — because a determinant of overlaps is blind to the unitary mixing a degenerate eigensolver leaves, *and* because the Fukui-Hatsugai-Suzuki lattice sum is an exact integer on any mesh where a Riemann sum of a pointwise curvature is not.

Z2 has two independent methods and running both is the check. Where they disagree the parity route is the answer, because it has no mesh and the Wilson route does WannierFlow.gap_step is the Wilson result's own diagnostic and is the number to read before believing the integer.

Two things bite in a plane-wave code and both are silent: neighbouring k-points do not share a G-sphere, so coefficients are aligned by Miller index and the wrap at the zone edge is a *shift* of that index, without which the Chern number comes out smooth and non-integer.

The equations

Everything is built from the overlap of occupied manifolds at neighbouring k-points, $M_{mn}^{(\mathbf{b})} = \langle u_{m\mathbf{k}} | \hat{S} | u_{n,\mathbf{k}+\mathbf{b}} \rangle$. The Berry curvature is the phase of a plaquette of them and the Chern number their lattice sum, which is an **exact integer on any mesh**:

$$F_{12}(\mathbf{k}) = \ln [U_1(\mathbf{k})U_2(\mathbf{k}+\hat{1})U_1(\mathbf{k}+\hat{2})^{-1}U_2(\mathbf{k})^{-1}], \quad C = \frac{1}{2\pi i} \sum_{\mathbf{k}} F_{12}(\mathbf{k})$$

Z_2 has two independent routes — the flow of Wannier charge centres, and, when there is an inversion centre, the parity product over the eight TRIM points:

$$(-1)^\nu = \prod_{i=1}^8 \prod_{n \text{ occ}}^{\text{Kramers}} \xi_{2n}(\Gamma_i)$$

```
from pypresso.workflows.topology import run_z2, run_berry_curvature
```

```
z2 = run_z2(system, pseudos, scf.density, axis=2)      # Wilson loop by default
print(z2.z2, z2.gap_step)    # read gap_step before believing the integer
chern = run_berry_curvature(system, pseudos, scf.density, shape=(12, 12))
print(chern)                # an exact integer on any mesh
```

9. Things with no pw.x counterpart

- **Spin spirals** (spiral_q, Elk's vqlss) by the generalized Bloch theorem: the up component lives at $\mathbf{k} + \mathbf{q}/2$ and the down at $\mathbf{k} - \mathbf{q}/2$, each on its own

plane-wave sphere. In the rotated frame the density is lattice periodic, so the SCF, the functional and the mixer are untouched. **Refused:** spin-orbit coupling permanently (it breaks the theorem, and Elk refuses it too), symmetry until the spin space group is written (so a spiral needs nosym), and ultrasoft/PAW.

- **Relaxing the spiral wavevector** (relax_spiral_q): $\mathbf{q}$ is a coordinate like an atomic position, $dE/d\mathbf{q}$ is `jax.grad` at frozen wavefunctions, and BFGS walks it downhill on the reciprocal metric. A magnetic field is refused here — its energy is outside the reported total, so the state is stationary for a different functional than the one being differentiated.
- **Per-atom magnetic fields** through a LOCAL_MAGNETIC_FIELDS card.
- **Topological invariants** (§8).
- **Rank-n tensor symmetrisation** — `symme.f90`'s `symmatrix3/symtensor3` written at any rank.

The spin-spiral equations

The generalized Bloch theorem: a spiral of wavevector $\mathbf{q}$ is a spinor whose two components live on *different* plane-wave spheres,

$$\psi_{\mathbf{k}}(\mathbf{r}) = \begin{pmatrix} e^{i(\mathbf{k}-\mathbf{q}/2)\cdot\mathbf{r}} u^{\uparrow}(\mathbf{r}) \\ e^{i(\mathbf{k}+\mathbf{q}/2)\cdot\mathbf{r}} u^{\downarrow}(\mathbf{r}) \end{pmatrix}$$

so in the rotated frame the density is lattice periodic and the SCF, the functional and the mixer are untouched. Only two terms carry $\mathbf{q}$ — $|\mathbf{k} \mp \mathbf{q}/2 + \mathbf{G}|^2$ and v_{nl} — and $dE/d\mathbf{q}$ at frozen coefficients is `jax.grad` of them.

```
from pypresso.workflows.spiral import run_spiral_scan, relax_spiral_q
```

```
scan = run_spiral_scan(system, pseudos, wavevectors)    # E(q)
best = relax_spiral_q(system, pseudos)                  # BFGS on the reciprocal metri
print(best.wavevector, best.scf.total_energy)
```

10. Performance: dials, memory and GPUs

Two batching dials control how much is in flight, and both default to QE's loop — one k-point and one band at a time, which is what a *cache* wants:

```
PYPRESSO_K_BATCH=all PYPRESSO_BAND_BATCH=all python3 run.py
```

or per call, `run_scf(..., k_batch=...)`. `all` means one `vmap` over the whole axis. The chunk size changes the order contributions are summed in and nothing else — round-off, $\sim 1\text{e-15}$ Ry.

On a GPU both defaults are wrong. A GPU has no cache to fit in and inverts the conclusion: on a ten-atom metal, `k=1, b=1` costs **4.5x** what `k=all, b=all` does on the same card. Set both. The plausible half-measure `k=all, b=1` is the *worst* setting available, because it buys the batched mode's memory with the looped mode's kernel launches.

Measured GPU speedups (one H200 against one CPU core, per SCF iteration): 2x on a cheap metal, 3–5x on norm-conserving silicon, 9–21x with an augmentation charge or a magnetization, and **83–115x on spin-orbit**. Energies agree to $\sim 1\text{e-9}$ Ry

— the same agreements the CPU already has. See `performance/gpu-sweep.pdf` and `PERFORMANCE.md`.

Memory. Spin-orbit is where it bites: a 20-atom bismuth cell with a relativistic ultrasoft dataset peaks at **34.7 GB**, which fits a 141 GB card and does not fit a 32 GB one. Everything else in that sweep sits under 2 GB.

Precision. Every dtype comes from `config.Precision` x64 is enabled before any array exists, and all validation is float64. float32 is a performance mode and **never one a correctness claim is made in**.

11. Accuracy summary

Against Quantum ESPRESSO on the same input:

quantity	agreement
total energy, norm-conserving LDA	$\sim 1\text{e-}9$ Ry
ultrasoft, PAW	$\leq 3\text{e-}9$ Ry
PBE / revPBE / PBEsol	$\leq 6\text{e-}9$ Ry
metals, every smearing and tetrahedra	$2.5\text{e-}8$ Ry
collinear spin	$1.2\text{e-}9$ Ry
noncollinear magnetism	$2.8\text{e-}9$ Ry
spin-orbit coupling	$\leq 1.3\text{e-}8$ Ry
DFT+U	$\leq 6.7\text{e-}9$ Ry
band structure	0.0002 eV
forces	$\leq 2\text{e-}5$ Ry/bohr
stress (13 cases)	$\leq 2.7\text{e-}7$ Ry/bohr ³
relaxed geometry	$1\text{e-}6$ bohr
dielectric constant (NC, US, PAW)	$\leq 1.2\text{e-}4$
Born effective charges (NC)	every printed digit
phonons at Γ , silicon	0.05 cm^{-1}
phonons at Γ , aluminium (metal)	0.002 cm^{-1}
Raman/IR spectra vs <code>dynmat.x</code>	every printed digit

One case does not close and is recorded rather than explained: `bil0-soc`, ten bismuth atoms with a fully-relativistic dataset, sits **$1.9\text{e-}4$ Ry** from QE on a total of -1477.737 — $1.3\text{e-}7$ relative. Both codes choose the same symmetry operations, the same k-point, the same grid and the same G-vectors, and both converge. It is in the test suite at that tolerance, not hidden.

Where to go next

- `README.md` — the short tour and a first calculation
- `notebooks/` — 26 worked examples, each on a concrete system, executed and committed with a `.md` export beside it
- `PERFORMANCE.md` — the running performance log, CPU and GPU
- `PLAN.md` — the architecture, the phase breakdown, and the trap catalogue
- `GPU.md` — the GPU roadmap